Learning to Get Back Up: Fall Recovery for the MuJoCo Ant

CS 839 Programming Assignment · Shivam Mittal · Fall 2026

Summary

- **Task:** the Ant starts fallen over (on its side or upside down, sometimes dropped, on flat, bumpy or sloped ground, sometimes shoved) and must get back on its feet and stay standing.
- **Success:** standing upright and still for the last 2.5 s of a 20 s episode.
- **Result:** SAC learns it reliably. All 5 seeds learn, and after training the 5 policies succeed in 99% of 500 new test episodes (100 per seed, the same kind of falls as in training, no exploration randomness). PPO mostly does not: 2 of 5 seeds learn, and it succeeds in 17% of the same 500 episodes, even with 5× more training data.
- **Robustness:** SAC also recovers on rubble it never trained on, after 3 m drops, on ice and on slopes up to 35° (often by sliding to flatter ground first). It fails mainly when two legs stop working or the motors are very weak.

1. The behavior: self-righting

What. After a fall, the Ant must roll back onto its feet and then stand still.

Why I chose it.

- A legged robot that cannot get up after a fall needs a person to come and stand it up. In disaster zones, mines or rough outdoor terrain, that person would have to walk into a dangerous area.
- Scripted get-up motions fail on unusual poses and uneven ground, which is why learned recovery is an active research topic (e.g. FR-Net, 2025). I plan to continue this toward a real robot.

Why it is hard for the Ant.

- Its hips only swing sideways (±30°) and its knees only bend one way. Upside down, its legs point at the sky and its round torso rests on the ground.
- To flip, it has to throw its legs to build momentum and use brief ground contacts. This is a very different motion from walking, and it must then stop moving and balance.
- Doing nothing never works (0%, or 8% when a drop lands it on its feet by luck). Random actions never work (0%).

2. The MDP

The environment is `AntRecoveryLowCost-v0`, in `robot_recovery/ant_recovery.py` (`AntRecoveryEnv`, settings in `RecoveryConfig`). It reuses Gymnasium's Ant body and motors unchanged and adds a terrain strip to the XML (`robot_recovery/assets/ant_recovery.xml`). My Gymnasium version (0.29.1) has no `Ant-v5`, so I used the `Ant-v4` model, as the assignment allows.

Action space. Unchanged: 8 torques in $[-1, 1]$, one per joint (gear 150). Each episode scales them by a random motor-strength factor.

State (observation), 33 numbers. I only used what a real robot can measure:

- gravity direction in the torso frame (3), which is what an IMU reports
- torso angular velocity (3) and linear velocity in the torso frame (3)
- joint angles (8) and joint velocities ×0.1 (8)
- the previous action (8)

Compared to the default Ant observation, I added torso orientation and removed absolute height and position, which a real robot does not know.

Is it Markov? No.

- The true state also includes things the Ant cannot observe: the terrain under it, friction, payload, motor strength, its height above the ground, and whether a shove is coming.
- So two identical observations can have different futures (for example, the same pose on ice vs. on grippy ground).
- This makes it a partially observable MDP (POMDP). The previous action adds a little short-term memory. I kept the policy memoryless on purpose, because a real robot's controller would see the same limited sensor data.

Start of each episode (`reset_model`):

- **Terrain:** flat (50%), bumpy (25%, bumps up to 6, 10 or 14 cm) or a slope (25%, with 5°, 10° or 15° faces).
- **Pose:** random heading, tilted 120–180° (side to fully upside down), random joint angles.
- **Drop:** 30% of episodes start 0.3–1.0 m higher, spinning at up to 3 rad/s.
- **Random physics:** friction ×0.8–1.2, torso mass ×0.9–1.3, motor strength ×0.9–1.1.
- **Shove:** in 50% of episodes, a sudden push of 0.3–1.0 m/s (plus spin) at a random time between 2 s and 15 s.

Reward. Given every step to guide learning. It is not the success metric. With c = how upright the torso is (1 = fully upright, −1 = upside down), h = torso height above the ground, ω = torso spin speed:

$$r = \left(\frac{c+1}{2}\right)^2 + 0.5 \cdot \text{clip}\left(\frac{h-0.25}{0.3}, 0, 1\right)\mathbb{1}[c > 0.7] + 0.5 \cdot \mathbb{1}[c > 0.9]e^{-\omega^2/2} - 0.005\ \|a\|^2 - 0.01\ \|a - a_{\text{prev}}\|^2$$

- Term 1 rewards being upright.
- Term 2 rewards standing tall (the torso only rises above 0.25 m when the legs hold it up).
- Term 3 rewards holding still once upright.
- The last two penalize effort and jerky actions, which matters on real motors.

`AntRecovery-v0` doubles the two penalty weights (0.01, 0.02). Halving them was the most important change I made for PPO (see Section 5).

Episode length and termination.

- 400 steps of 0.05 s = 20 s (physics at 100 Hz).
- I removed the default Ant termination on torso height, since it would end every episode at the start for a fallen robot.
- The only termination is a broken (non-finite) simulator state, which never happened. Episodes end at the time limit. CleanRL's SAC handles this time-limit cut-off correctly. CleanRL's PPO treats it as a real ending.

3. True performance metric

Success is one number per episode, 1 or 0. It is 1 if the Ant is upright and steady at **every step of the last 2.5 s** (the last 50 of the 400 steps), and 0 otherwise. Upright and steady means all three hold:

1. $c \geq 0.9$, where c is the dot product of the torso's up axis with the ground's normal direction, i.e. the torso is within 25.8° of it. The normal is computed once, at the start point, from ground heights 0.15 m either side in x and y. It is straight up on flat ground and rubble.
2. $h \geq 0.32$ m above the ground directly below (lying on the torso gives ≈0.25 m, standing gives ≈0.40 m)
3. $\omega \leq 1$ rad/s torso spin.

The environment computes it at the end of every episode and returns it in `info`. During training CleanRL logs it for every finished episode (`charts/success`), which gives the success curves in Section 4. I report the **success**

rate, the average of this 0/1 value over episodes. Secondary: **time to recover**, the first time the Ant stays upright and steady for 1 s in a row (20 s if never).

Why this metric.

- It measures the actual goal, ending on its feet and still, and makes sense without knowing the reward. For example, the Ant recovered in 95% of falls.
- Things the reward partly pays for do not count: being half-rolled over, standing up briefly and falling again, or wobbling while upright.
- It also requires **staying** up after a shove.

A limitation I found later. Being upright is measured against the ground direction at the start point. If the Ant slides off a steep slope and stands on flat ground, it is standing straight but tilted relative to the start slope, and on slopes above 26° it can never pass. I kept the metric as defined for all main results and measured the size of this effect separately (Section 7).

4. Learning curves (PPO vs. SAC)

Training setup. Both algorithms used CleanRL's default settings, apart from the PPO tuning in Section 5.

	SAC	**PPO**
Environment steps per run	1M	5M
Training episodes per run (400 steps each)	2,500	12,500
Runs	5 (seeds 1–5)	5 (seeds 1–5)
Training time per run (1 CPU core, CHTC)	6.1–7.7 h	2.3–3.8 h
Networks	256-256, ReLU (actor and two critics)	64-64, tanh (actor and critic)
Learning rate	3e-4 actor, 1e-3 critic	3e-4, decayed to 0
Batch	256 samples from a 1M-step replay buffer	2,048 steps, 32 minibatches, 10 epochs
Other	$\gamma = 0.99$, $\tau = 0.005$, entropy weight tuned automatically, learning starts after 5k steps	$\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2, no entropy bonus, observations and rewards normalized

Table 1: Final training runs. Times are measured from the training logs.

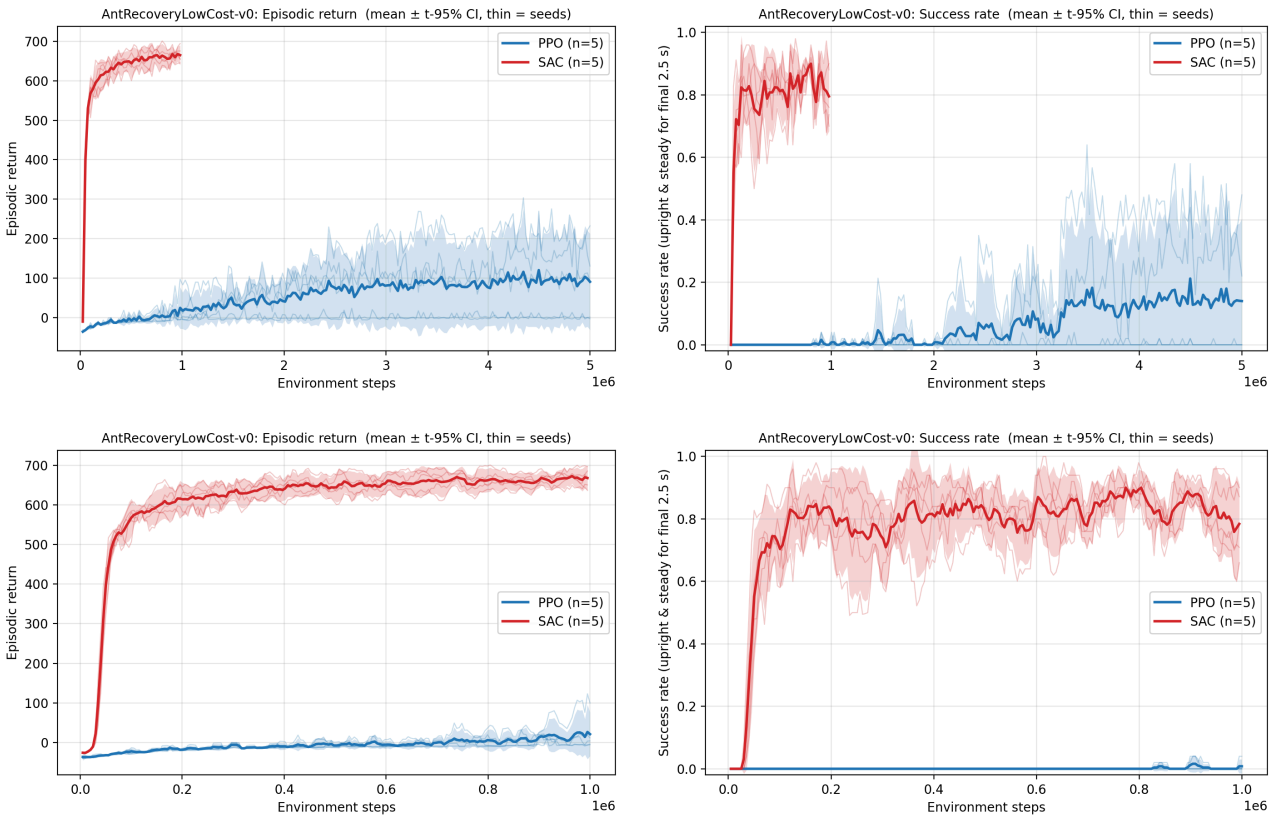


Figure 1: Episodic return (left) and true metric, success rate (right) vs. environment steps on `AntRecoveryLowCost-v0`. Top: full runs (SAC 1M steps, PPO 5M). Bottom: the first 1M steps, for an equal-budget comparison. Bold line: mean over 5 paired seeds (seeds 1–5 for both algorithms). Band: Student-t 95% interval $\bar{x} \pm 2.78s/\sqrt{5}$. Thin lines: individual seeds.

Plotted quantities.

- **x-axis:** environment steps taken so far in training (not updates or wall-clock time).
- **Return plot:** the total reward of one training episode (sum over its 400 steps).
- **Success plot (the true metric):** the share of training episodes that succeeded. Each episode is 0 or 1, so after smoothing over 50 episodes a value of 0.8 means 40 of the last 50 episodes succeeded.
- These are training episodes, so SAC and PPO are still exploring with random actions.

How I built the curves.

- Each run logs one value per finished training episode, at irregular steps.
- For each seed, I first smooth with a trailing 50-episode moving average.
- I then linearly interpolate each seed onto 200 evenly spaced environment steps (no extrapolation past a run's end).
- Only then do I average across seeds (smooth first, then aggregate). The mean and band are drawn only where all 5 seeds have data.

Results on training episodes (mean ± 95% half-width, per-seed values in brackets):

- **SAC:** 76 ± 20% success by 0.25M steps and **80 ± 11%** at 1M [76, 90, 74, 88, 70]. Return 665 ± 25. Average time to recover 3.7 ± 0.5 s. All 5 seeds learn.
- **PPO at 1M: 0.8 ± 2.2%** [0, 4, 0, 0, 0], return 17 ± 46.
- **PPO at 5M: 14 ± 26%** [48, 22, 0, 0, 0], return 91 ± 122. Only 2 of 5 seeds ever learn to get up. The other 3 stay on their backs. The wide band describes this split badly, which is why the per-seed lines matter.

- Return and success rise together for both algorithms, so the policies are not exploiting the reward without getting up.
- SAC's training curve levels off near 80% because it explores with random actions during training. Run without that randomness, the same five SAC policies succeed in **99%** of 500 new test episodes drawn the same way as training episodes (Table 3, training mix row).

5. Comparison and tuning

Which learned better? SAC, on all three axes:

- **Final performance:** SAC reaches 80% success during training and 99% on the 500 test episodes of the training-mix row of Table 3. PPO reaches 14% and 17% on the same measures, even though it got 5× the data.
- **Sample efficiency:** at 1M steps, SAC is at 80% and PPO at 0.8%. SAC passes 50% within about 50k steps. PPO's best seed needs 2.5–3M steps to reach 20%.
- **Wall-clock time:** SAC is much slower per step (about 40 vs. 450 steps per second on one CPU core), because it does a gradient update after every step. Its 1M steps took 6–8 h per run, against 2–4 h for PPO's 5M steps. Even so, SAC reached 50% success within about 20 minutes, which PPO never reached.
- **Variance across seeds:** SAC's seeds end between 70% and 90% (97–98% at test). PPO splits into two seeds that learn and three that never do, so for PPO the seed matters more than the settings.

Why I think SAC wins.

1. **The first flip is rare, and only SAC can reuse it.** Early on, a successful flip happens in only a few episodes. SAC keeps every step it has taken in a 1M-step replay buffer and trains on samples from it after every step, so it learns from a rare success many times. PPO trains on its latest 2,048 steps for 10 passes and then throws them away, so a lucky flip is quickly forgotten.
2. **An easy trap.** Lying still on the back costs nothing, and being partly tilted earns little. PPO quickly stops exploring and settles on doing nothing. Adding an entropy bonus (extra reward for acting randomly) made PPO worse in my tests. SAC is built to keep its actions as random as possible while still earning reward (its maximum-entropy objective, with an automatically tuned weight), so it keeps exploring until it finds the flip.
3. **Not network size.** CleanRL's PPO uses small 64-unit networks and SAC uses 256-unit ones. I tried 256-unit networks for PPO and they did not help.
4. **Time limits (smaller effect).** CleanRL's PPO treats the 20 s cut-off as a real ending, which undervalues still standing at the end. CleanRL's SAC handles the cut-off correctly.

Tuning. SAC used CleanRL's defaults throughout. For PPO I ran 28 tuning runs of 5M steps each (job lists in `chtc/batches/ppotune*.txt`). Success at 5M steps (average of the last 100 training episodes), mean [per seed]:

Reward	Envs × steps	Entropy	Width	Success at 5M
`v0` (costs 0.01/0.02)	1×2048	0	64	0% [0, 0]
`v0`	1×2048	0.01	64	4% [8, 0]
`v0`	16×256	0 / 0.01	64	0% (all four seeds)
low-cost	**1×2048**	**0**	**64**	**35% [46, 23]**, chosen (CleanRL defaults)
low-cost	1×2048	0.01	64	0% [0, 0]
low-cost	16×256	0	64	24% [49, 0]
low-cost	16×256	0.01	64	10% [17, 2]
low-cost	1×2048	0	256	17% [0, 23, 27]
low-cost	1×2048	0.01	256	0% [0, 0, 0]
low-cost	16×256	0	256	20% [59, 0, 0]
low-cost	16×256	0.01	256	8% [0, 25, 0]

Table 2: PPO tuning (28 runs). The final PPO setting reproduces its tuning runs exactly for seeds 1–2.

- **What worked:** halving the effort and jerkiness penalties. With the original weights, lying still is the cheapest behavior, and PPO never escaped it in 5M steps. SAC also improved (88% → 95% in single-seed pilots), so I used the lower penalties for both algorithms.
- **What did not work:** an entropy bonus of 0.01, 16 parallel environments with shorter rollouts, and 4× wider (256-unit) networks.

6. Robustness beyond training

Because my goal is a real robot, I tested every trained policy without action randomness in 12 conditions (100 episodes each, the same episode seeds for every policy), along with two reference policies.

- Each condition changes one factor from training.
- The hard condition combines several: unseen terrain, slopes up to 25°, 1–2 m/s shoves, 0.5–1.5 m drops, friction ×0.4–1.5, payload ×1–1.8 and motors ×0.7–1.1.
- The rubble field (70 tilted, partly buried bricks and slabs) never appears in training.

Condition	SAC	PPO	Do nothing	Random
Training mix	**99** (99–100)	17 (0–47)	0	0
Flat	**100** (100–100)	16 (0–48)	0	0
Bumpy (≤14 cm)	**100** (100–100)	14 (0–37)	0	0
Slopes 5–15°	**97** (93–100)	14 (0–41)	0	0
Rubble (unseen)	**100** (100–100)	13 (0–42)	0	0
Shove 1–2 m/s	**99** (98–100)	15 (0–43)	0	0
Drop 1–1.5 m, spinning	**99** (99–100)	22 (0–44)	8	0
Friction ×0.3–0.5	**99** (98–100)	7 (0–23)	0	0
Payload ×1.5–2	**99** (99–100)	12 (0–35)	0	0
Motors ×0.6–0.7	**99** (98–100)	11 (0–45)	0	0
One leg dead	**91** (77–98)	1 (0–3)	0	0
Hard (combined)	**92** (88–100)	12 (0–37)	2	0

Table 3: Success (%) without action randomness: mean over the 5 seeds (range over seeds), 100 episodes per condition and seed. Doing nothing only succeeds when a drop happens to land the Ant on its feet.

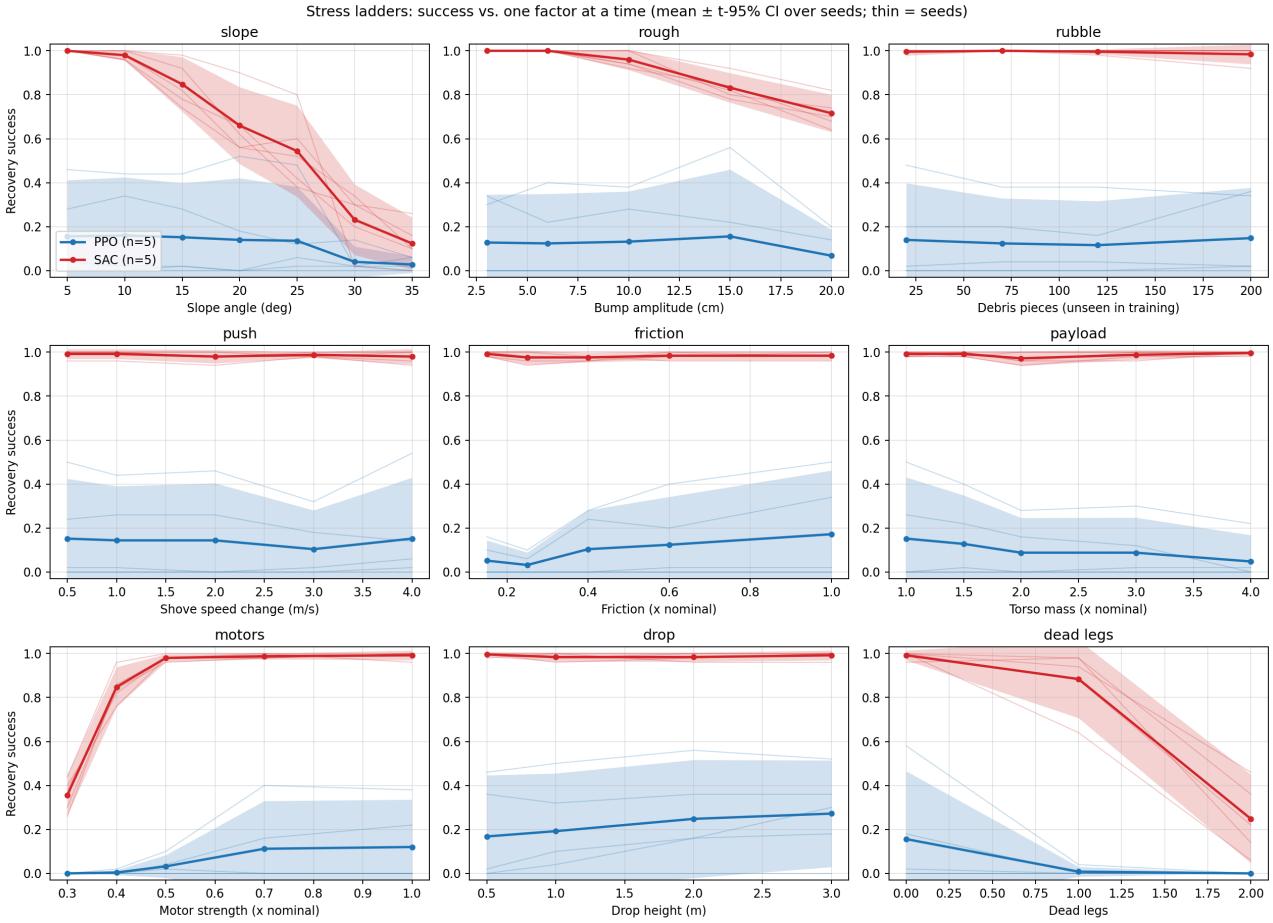


Figure 2: Stress tests. Each of the 9 panels fixes one factor at increasingly hard levels (x-axis), while everything else is randomized as in training. The y-axis is success without action randomness. Red: SAC, blue: PPO. Bold line and band: mean and t-95% interval over the 5 seeds, thin lines: individual seeds. 50 episodes per level and seed, the same episodes for every policy. The rubble field is never seen in training.

Where recovery breaks (Figure 2, SAC, mean over 5 seeds):

- **No effect** (≥97% at every level): drops up to 3 m, shoves up to 4 m/s, friction down to ×0.15 (ice), payload up to ×4, rubble up to 200 pieces.
- **Slopes:** ≥98% up to 10°, 85% at 15° (the edge of training), then 66%, 54%, 23%, 12% at 20°, 25°, 30°, 35°. Most of this drop is the metric limitation described in Section 3, not a real failure (Section 7).
- **Bumps:** 100% up to 6 cm amplitude, 96% at 10 cm (bumps up to 20 cm tall), 72% at 20 cm (bumps up to 40 cm).
- **Weak motors:** ≥98% down to ×0.5, 85% at ×0.4, 36% at ×0.3.
- **Dead legs:** 99%, 88%, 25% with 0, 1, 2 limp legs.
- **PPO** stays between 0% and 27% at every level.

Lost or weak motors, not impacts, weight or slopes, are what I need to fix before moving to a real robot, e.g. by training with simulated motor failures.

7. Failure cases

How SAC fails.

- In the main tests (Table 3), SAC failed 123 times out of 6,000 episodes. In 91% of those it never got up at all. In the other 9% it got up but was not steady at the end.

- **Two dead legs or motors at 30%:** the Ant cannot swing its legs hard enough to roll over and stays on its back (98% of these failures never got up). The video shows both.

How PPO fails. In the 3 seeds that never learn, the Ant stays on its back for the whole episode. 98.6% of PPO's failures never got up.

Steep slopes are mostly a metric problem. I replayed the exact same slope and bump episodes and scored them a second time, with the ground direction measured under the torso at every step instead of at the start point (`scripts/terrain_metric_check.py`). The original scoring reproduces the stress-test numbers exactly.

- On slopes, SAC succeeds in 93–100% at every angle up to 35° with this scoring (Figure 3).
- At 30°, 79% of episodes end on near-flat ground. The Ant usually slides or tumbles down, then stands up at the bottom. So it recovers on steep slopes, but often by getting off the slope rather than standing on it.
- On big bumps the gain is smaller (72% → 89% at 20 cm), so bumps are a real, smaller weakness.
- In the main tests, SAC goes from 97% to 100% on slopes and from 92% to 99% on the hard condition. PPO barely changes, because it rarely gets up at all.
- Training slopes only go up to 15°, so this affects the training curves very little.

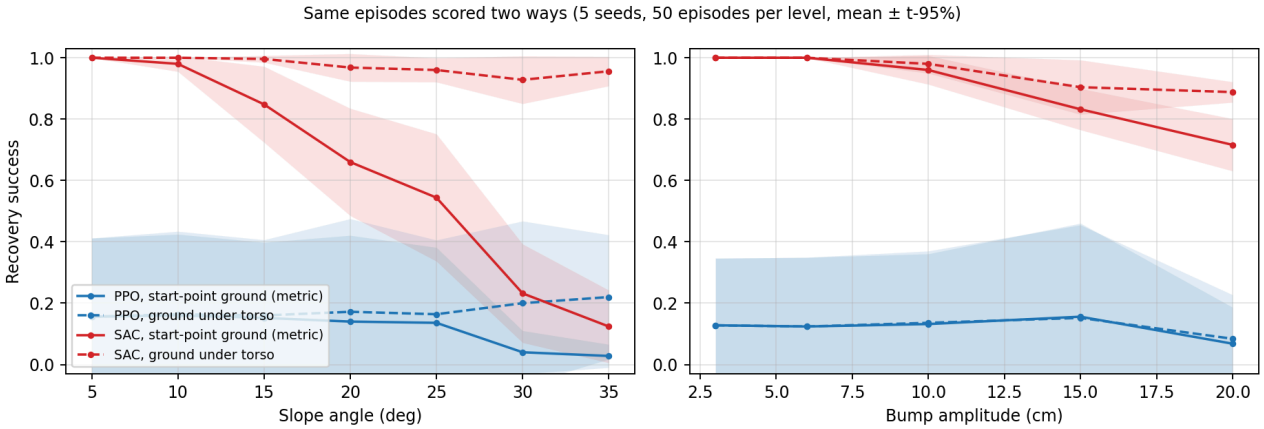


Figure 3: The slope and bump episodes of Figure 2, scored twice: with the ground direction at the start point (solid, the metric used everywhere else) and with the ground direction under the torso (dashed). Red: SAC, blue: PPO. Mean and t-95% interval over the 5 seeds, 50 episodes per level and seed.

8. Video

- **Main video:** `videos/best_policy_sac_seed4.mp4` (101 s). Best policy = SAC seed 4, picked by a rule I set in advance: highest average success over all 12 test conditions (98.3%). It has two parts.
- **Part 1 (37 s):** 8 consecutive, unselected training-style episodes (episode seeds 10000–10007). Captions give the terrain and the outcome by the true metric. All 8 recover, in 2.1–5.9 s. The policy rolls off its back by swinging its legs, plants all four feet, and holds still with its torso about 0.5 m high.
- **Part 2 (60 s):** one episode per harder condition (20° slope, bumps up to 20 cm, 200-piece rubble, 3 m drop, 4 m/s shove, ice, ×4 payload, one dead leg), each captioned with the policy's measured success rate there, plus two failures (motors at 30% strength, two dead legs). Every success clip is the first episode tried. Part 2 is also saved on its own as `videos/highlight_reel_sac_seed4.mp4`.
- Each clip ends 1.5 s after the Ant is steady (it keeps standing until 20 s).
- **About `--capture-video`:** I trained on a cluster without a display, so I recorded videos afterwards from the saved policy. `videos/recordvideo_sac_seed4/` has three full 20 s episodes recorded with Gymnasium's `RecordVideo`, the same recorder CleanRL's `--capture-video` uses. The captioned videos use a camera that follows the Ant sideways at a fixed height, so drops and slopes are visible.

9. Code and reproduction

What is in the archive:

- the environment (`robot_recovery/`) and the modified XML (`robot_recovery/assets/ant_recovery.xml`)
- my copies of the two CleanRL scripts (`algos/`) and the diff against CleanRL commit `fe8d8a0` (`patches/cleanrl.diff`)
- analysis scripts (`scripts/`) and `README.md`

How the code works.

- **Terrain:** one MuJoCo heightfield strip of six 4×4 m tiles (x from 4 to 28 m, 5 cm grid), three bumpy and three pyramids. Tile edges taper to zero height, so there are no cliffs. Flat-ground starts are in a 4×4 m area around the origin, away from the strip.
- **Placement:** after choosing the pose, the Ant is lowered until its lowest point is 2 cm above the highest ground within 1.2 m, so it never starts inside the ground.
- **Shove:** at the chosen step, a velocity change is added directly to the torso's velocity.
- **Friction:** MuJoCo uses the larger friction of two touching surfaces, so I scale the robot and the ground together.
- **Dead legs (test only):** both motor gains of the chosen leg are set to 0.
- **Rubble (test only):** 70 to 200 fixed boxes added to a temporary copy of the XML, at x = −10 m, away from everything used in training.
- **Metric logging:** the environment updates the success counter every step and reports it in `info`. My edit to CleanRL logs it when an episode ends.
- **Testing:** actions without randomness (PPO uses the Gaussian mean, SAC the tanh of the mean). Test episodes use seeds from 10,000 (main tests) and 20,000 (stress tests), separate from training, and every policy sees the same episodes.

My changes to CleanRL (no algorithm code changed):

- import my environment package
- log `success`, `recovered` and `time_to_recover` at the end of each episode
- save the SAC model, and save PPO's observation-normalization statistics with its model
- a PPO `--hidden-size` option (default 64, i.e. unchanged), used only in tuning

Final runs, for seeds $N = 1, \ldots, 5$:

```
python algos/sac_continuous_action.py --env-id AntRecoveryLowCost-v0 --seed N --total-timesteps
1000000 --save-model
python algos/ppo_continuous_action.py --env-id AntRecoveryLowCost-v0 --seed N --total-timesteps
5000000 --save-model
```

`README.md` lists the setup, the tuning commands, and the exact commands for every figure, table and video. All runs used one CPU core each on UW–Madison's CHTC cluster (no GPU).

10. Limitations

- Everything is in simulation, and the Ant is not a real robot design. A real robot has motor delays, sensor noise and different leg shapes, so these results do not transfer directly.
- The success metric measures being upright against the ground at the start point, which undercounts recovery on steep slopes (Section 7).
- 5 seeds per algorithm is small. PPO's split between seeds that learn and seeds that do not makes its average especially uncertain.
- PPO might do better with far more data (for example with GPU simulation), which I could not afford on CPUs.

- The policy has no memory, so it cannot learn hidden properties such as friction or payload during an episode.
- The videos were recorded after training from the saved policy, not during training.

11. Use of AI

I used Claude to survey related work on legged-robot fall recovery, brainstormed the environment (terrain, randomization,reward and metric), write the cluster job scripts, to launch and monitor the training runs, and took small help to write the evaluation, plotting and video scripts and also took help in writing a first draft of the report. I take full responsibility for the submitted work.